

## CURSO DE PYTHON

### CAPÍTULO 2

# Entradas y Salidas en Python

De la interacción básica con el usuario a la construcción de programas prácticos

|                 |                                         |
|-----------------|-----------------------------------------|
| <b>Nivel</b>    | Desde principiante hasta nivel práctico |
| <b>Enfoque</b>  | Desarrollo de Software Python           |
| <b>Lenguaje</b> | Python 3                                |

Material diseñado para comprender, practicar y aplicar el tema sin depender de otra fuente.

# Contenido del capítulo

- 1 Objetivo del capítulo
- 2 Introducción: entrada, proceso y salida
- 3 Conceptos fundamentales
- 4 `print()`: mostrar información
- 5 `input()`: recibir información
- 6 Conversión de datos
- 7 f-strings y formato de resultados
- 8 Ejemplos progresivos
- 9 Comparaciones y errores de confusión
- 10 Errores comunes y buenas prácticas
- 11 Conectando conocimientos
- 12 Ejercicios prácticos
- 13 Mini proyecto
- 14 Autoevaluación
- 15 Respuestas y explicaciones
- 16 Resumen y mapa de aprendizaje

# 1. Objetivo del capítulo

Este capítulo explica cómo un programa de Python se comunica con su entorno. La entrada permite recibir información; el proceso transforma o analiza esa información; y la salida comunica resultados, mensajes, reportes o estados.

## Al finalizar este capítulo serás capaz de:

- Explicar la diferencia entre entrada, proceso y salida.
- Usar `print()` para mostrar texto, números, variables y resultados.
- Usar `input()` para solicitar información al usuario.
- Reconocer que `input()` devuelve texto.
- Convertir datos con `int()`, `float()`, `str()` y `bool()`.
- Construir mensajes claros con f-strings.
- Combinar entradas, variables, operadores y salidas.
- Detectar errores de tipos y entradas inválidas.
- Crear programas relacionados con máquinas, sensores, inventarios y producción.

## Resultado esperado

Podrás construir programas interactivos que soliciten datos, realicen cálculos y presenten resultados ordenados y comprensibles.

# 2. Introducción: programas que reciben y producen información

Un programa real casi siempre necesita comunicarse. Un sistema de inventario recibe la cantidad de piezas y muestra un reporte. Un monitor industrial recibe una temperatura y comunica el valor medido. Una aplicación empresarial solicita datos de un cliente y confirma el registro.

Analogía: una máquina expendedora recibe una selección y un pago, procesa la solicitud y entrega un producto o un mensaje. En Python, el patrón es:

| Entrada           | Proceso                | Salida                      |
|-------------------|------------------------|-----------------------------|
| Datos que llegan  | Operaciones y análisis | Información que se comunica |
| Temperatura: 82.5 | Comparar o calcular    | "Temperatura alta"          |

Sin entradas, muchos programas dependerían de datos escritos directamente en el código. Sin salidas, el usuario no conocería el resultado. Por eso este tema conecta variables, tipos de datos, conversiones y operadores con aplicaciones reales.

# 3. Conceptos fundamentales

| Concepto | Definición                         | Ejemplo                |
|----------|------------------------------------|------------------------|
| Entrada  | Información que llega al programa. | El usuario escribe 25. |

| Concepto    | Definición                                       | Ejemplo                    |
|-------------|--------------------------------------------------|----------------------------|
| Proceso     | Operaciones que transforman o analizan datos.    | Calcular $25 + 10$ .       |
| Salida      | Información que el programa muestra o entrega.   | Mostrar "Resultado: 35".   |
| Interacción | Comunicación entre programa y persona o sistema. | Solicitar una temperatura. |

## Flujo general

```

Usuario o sistema
|
v
ENTRADA DE DATOS
|
v
PROCESAMIENTO
|
v
SALIDA DE RESULTADOS
|
v
Usuario o sistema recibe la respuesta

```

El flujo puede repetirse. Más adelante, los ciclos permitirán solicitar y procesar datos continuamente.

## 4. La función print(): mostrar información

### ¿Qué es?

`print()` es una función incorporada de Python que envía información a la salida estándar, normalmente la consola. Una función es un bloque de comportamiento que se ejecuta mediante su nombre y paréntesis.

### ¿Para qué se utiliza?

- Mostrar mensajes e instrucciones.
- Presentar valores de variables.
- Comunicar resultados de cálculos.
- Mostrar estados, advertencias y reportes.
- Observar valores durante una prueba.

### Sintaxis

```
print(valor)
```

El valor puede ser texto, un número, una variable o una expresión.

### Ejemplo y análisis

```

print("Hola, Python")
print(25)
nombre = "Ana"
print(nombre)
print(10 + 5)

```

- 1 La primera línea muestra texto.
- 2 La segunda muestra un entero.
- 3 La tercera crea una variable llamada nombre.
- 4 La cuarta muestra el contenido de nombre.
- 5 La quinta calcula  $10 + 5$  y muestra 15.

### Varios valores, sep y end

```
print("Nombre:", "Luis")
print("2026", "09", "05", sep="-")
print("Cargando", end="...")
print("Listo")
```

sep cambia el separador entre valores. end cambia lo que se escribe al final de print(); por defecto es un salto de línea.

## 5. La función input(): recibir información

### ¿Qué es?

input() muestra una solicitud opcional, pausa el programa y espera a que el usuario escriba algo. Cuando se presiona Enter, devuelve lo escrito como texto.

### Sintaxis

```
variable = input("Mensaje para el usuario: ")
```

### Funcionamiento paso a paso

- 1 Python muestra el mensaje.
- 2 El usuario escribe un valor.
- 3 El usuario presiona Enter.
- 4 input() devuelve una cadena de texto.
- 5 La variable guarda esa cadena.
- 6 El programa continúa ejecutándose.

### Ejemplo

```
nombre = input("Escribe tu nombre: ")
print("Hola", nombre)
```

Si el usuario escribe Carlos, la variable nombre contiene "Carlos" y la salida será Hola Carlos.

### Regla esencial

```
edad = input("Edad: ")
print(type(edad))
```

Aunque se escriba 25, type(edad) será str. La consola recibe caracteres; para realizar cálculos se necesita convertir el texto.

## 6. Conversión de datos

Una conversión transforma un valor de un tipo a otro. Es necesaria porque `input()` devuelve `str`, incluso cuando el usuario escribe números.

### `int()`: entero

```
edad = int(input("Edad: "))
print(edad + 1)
```

`int()` interpreta el texto como un número entero. Con entrada 20, `edad` contiene 20 y el resultado es 21.

### `float()`: decimal

```
temperatura = float(input("Temperatura: "))
print(temperatura + 0.5)
```

`float()` es adecuado para temperaturas, costos, pesos, velocidades y horas.

### `str()`: texto

```
numero = 125
mensaje = "Piezas: " + str(numero)
print(mensaje)
```

`str()` permite convertir un número en texto para construir una cadena.

### `bool()`: verdadero o falso

```
print(bool(1))      # True
print(bool(0))      # False
print(bool(""))     # False
print(bool("Python")) # True
```

Una cadena no vacía es verdadera, incluso si contiene la palabra "False". Por eso `bool()` debe usarse con criterio.

## Errores de conversión

```
int("veinte")      # ValueError
int("12.5")         # ValueError
"Total: " + 50      # TypeError
```

Usa `float()` para decimales y f-strings o `str()` para combinar texto y números.

## 7. f-strings y formato de resultados

Una f-string es una cadena prefijada con `f` que permite insertar variables o expresiones dentro de llaves. Es clara, legible y muy útil para reportes.

```
nombre = "Marta"
piezas = 120
print(f"Operador: {nombre} | Piezas: {piezas}")
```

Python calcula lo que está dentro de `{ }` y lo inserta en el texto.

```
temperatura = 27.4567
costo = 1250.5
print(f"Temperatura: {temperatura:.2f} °C")
print(f"Costo: ${costo:,.2f}")
```

`:.2f` muestra dos decimales. Mostrar unidades y formato facilita interpretar los resultados.

## 8. Ejemplos progresivos

## Nivel 1 — Saludo interactivo

```
nombre = input("Nombre: ")
print(f"Bienvenido, {nombre}.")
```

Recibe un dato y lo incorpora en un mensaje.

## Nivel 2 — Suma de dos números

```
numero_a = int(input("Primer número: "))
numero_b = int(input("Segundo número: "))
suma = numero_a + numero_b
print(f"La suma es: {suma}")
```

La conversión ocurre antes de la suma. Sin `int()`, + concatenaría texto.

## Nivel 3 — Monitor de temperatura

```
maquina = input("Máquina: ")
temperatura = float(input("Temperatura: "))
limite = 80.0
margen = limite - temperatura
print(f"Máquina: {maquina}")
print(f"Temperatura: {temperatura:.1f} °C")
print(f"Margen: {margen:.1f} °C")
```

Este programa recibe datos industriales, calcula un margen y genera un reporte. En el capítulo de condicionales podrás convertir ese margen en un estado NORMAL o ALERTA.

## Nivel 4 — Registro de producción

```
maquina = input("Máquina: ")
turno = input("Turno: ")
buenas = int(input("Piezas buenas: "))
defectuosas = int(input("Piezas defectuosas: "))
total = buenas + defectuosas
print("\n===== REPORTE =====")
print(f"Máquina: {maquina}")
print(f"Turno: {turno}")
print(f"Total producido: {total}")
```

`\n` produce un salto de línea dentro del texto.

# 9. Comparaciones y errores de confusión

| Elemento             | Función                   | Ejemplo                        |
|----------------------|---------------------------|--------------------------------|
| <code>print()</code> | Muestra información.      | <code>print("Hola")</code>     |
| <code>input()</code> | Recibe texto.             | <code>nombre = input()</code>  |
| <code>int()</code>   | Convierte a entero.       | <code>int("25")</code>         |
| <code>float()</code> | Convierte a decimal.      | <code>float("25.5")</code>     |
| <code>str()</code>   | Convierte a texto.        | <code>str(25)</code>           |
| f-string             | Inserta valores en texto. | <code>f"Total: {total}"</code> |

## Errores de confusión comunes

- `print()` muestra, pero no guarda automáticamente.

- `input()` recibe texto, no un número.
- `print("nombre")` muestra la palabra; `print(nombre)` muestra el contenido.
- `2 + 3` produce 5; `"2" + "3"` produce `"23"`.
- Una f-string necesita la letra f antes de las comillas.

## 10. Errores comunes al trabajar con entradas y salidas

| Error                                             | Por qué ocurre                      | Corrección                          |
|---------------------------------------------------|-------------------------------------|-------------------------------------|
| Olvidar <code>int()</code> o <code>float()</code> | <code>input()</code> entrega texto. | Convertir antes de calcular.        |
| <code>int("3.5")</code>                           | El texto contiene decimales.        | Usar <code>float()</code> .         |
| <code>"Total: " + 50</code>                       | Texto y número no son compatibles.  | Usar <code>str()</code> o f-string. |
| Solicitud ambigua                                 | El usuario no sabe qué escribir.    | Indicar formato y unidad.           |
| Nombres como x                                    | No explican el dato.                | Usar nombres descriptivos.          |

### Buenas prácticas

- Escribe mensajes específicos: "Ingrese la temperatura en °C:".
- Convierte los datos inmediatamente cuando conozcas el tipo esperado.
- Usa nombres como `cantidad_piezas`, `costo_unitario` y `nombre_maquina`.
- Muestra etiquetas y unidades, no solo números aislados.
- Prefiere f-strings para reportes.
- Separa entrada, proceso y salida.
- Considera que una entrada puede ser inválida; más adelante aprenderás `try/except`.
- Calcula primero y muestra después cuando la expresión sea compleja.

### Seguridad y robustez

La entrada del usuario no debe asumirse correcta. Una entrada inesperada puede producir errores o resultados incorrectos. En capítulos posteriores aprenderás validaciones y manejo de excepciones.

## 11. Conectando conocimientos

| Conocimiento   | Relación                                                    |
|----------------|-------------------------------------------------------------|
| Variables      | Guardan datos recibidos y resultados.                       |
| Tipos de datos | Describen si el valor es texto, entero, decimal o booleano. |
| Conversiones   | Transforman el texto recibido al tipo necesario.            |
| Operadores     | Procesan los datos mediante cálculos.                       |

| Conocimiento | Relación                                                     |
|--------------|--------------------------------------------------------------|
| if y ciclos  | Más adelante validarán, decidirán y repetirán interacciones. |

## Ejemplo integrado

```

producto = input("Producto: ")
cantidad = int(input("Cantidad: "))
precio = float(input("Precio unitario: "))
total = cantidad * precio
print(f"Producto: {producto}")
print(f"Total: ${total:.2f}")

```

La secuencia completa es entrada → conversión → variables → operación → salida.

## 12. Ejercicios prácticos

No consultes las respuestas todavía. Intenta escribir, ejecutar y modificar cada programa.

### Ejercicio 1 — Comprensión básica

**Objetivo:** practicar print() y variables.

**Enunciado:** guarda tu nombre, ciudad y lenguaje favorito; después muestra una presentación.

**Requisitos:** tres variables y tres salidas.

**Pista:** asigna primero y muestra después.

### Ejercicio 2 — Aplicación

**Contexto:** una tienda necesita calcular una compra.

**Requisitos:** solicitar producto, cantidad y precio; convertir tipos; calcular total; mostrar recibo.

**Restricción:** usa variables en el cálculo.

**Pista:** total = cantidad \* precio\_unitario.

### Ejercicio 3 — Integración

Solicita empleado, horas trabajadas y pago por hora. Calcula el pago bruto y muestra un reporte. Combina input(), conversiones, variables, operadores y f-strings.

### Ejercicio 4 — Caso práctico industrial

Solicita máquina, turno, piezas buenas y defectuosas. Calcula el total producido y muestra un reporte con títulos y unidades. Las cantidades deben ser enteros.

### Ejercicio 5 — Reto

Crea un registro de consumo energético. Solicita voltaje, corriente, horas de operación y costo por kWh. Calcula potencia, energía y costo estimado. Usa conversiones correctas, nombres descriptivos, dos decimales y una salida profesional.

## 13. Mini proyecto

### Nombre: Registro de una estación de producción

**Contexto:** una estación industrial necesita registrar información de una máquina y generar un reporte de turno.

**Objetivo:** construir un programa que reciba datos, los procese y entregue un reporte claro.

**Requisitos funcionales:** solicitar máquina, operador, piezas buenas, defectuosas, horas trabajadas y meta; calcular total y diferencia respecto a la meta; mostrar un reporte.

**Requisitos técnicos:** usar `input()`, `int()`, `float()`, variables descriptivas, operadores y f-strings; no usar todavía `if`, `while`, `for` ni funciones definidas por el usuario.

## Paso 1 — Captura

Solicita y guarda todos los datos. Revisa que cantidades y horas tengan el tipo adecuado.

## Paso 2 — Procesamiento

Calcula producción total y diferencia con la meta. Decide qué valores son enteros y cuáles decimales.

## Paso 3 — Reporte

Organiza la salida con títulos, etiquetas y unidades. Mantén separadas captura, cálculo y presentación.

## Paso final — Pruebas

Prueba con cero defectuosas, producción superior a la meta y horas con decimales. Más adelante podrás agregar validaciones y estados.

# 14. Comprueba tus conocimientos

1. ¿Qué diferencia existe entre entrada, proceso y salida?
2. ¿Qué función muestra información en consola?
3. ¿Qué función recibe texto del usuario?
4. ¿Qué tipo devuelve `input()`?
5. ¿Qué ocurre al sumar `input()` directamente con un número?
6. ¿Cuál es la diferencia entre `int()` y `float()`?
7. ¿Qué muestra `print("nombre")` si `nombre` es una variable?
8. ¿Para qué sirve `str()`?
9. ¿Qué resultados producen `2 + 3` y `"2" + "3"`?
10. ¿Qué es una f-string?
11. ¿Qué significa `:.2f`?
12. Analiza `edad = int(input("Edad: "))`.
13. ¿Por qué `int("3.5")` falla?
14. ¿Por qué conviene usar nombres descriptivos?
15. ¿Qué problema existe en `print("Total: " + 50)`?
16. Escribe la estructura para solicitar un precio decimal.
17. ¿Por qué mostrar unidades?
18. Menciona dos errores comunes de entrada.
19. ¿Cómo se relacionan variables e interacción?
20. Describe el flujo de un programa que suma dos números.

## 15. Respuestas y explicaciones

1. La entrada proporciona datos, el proceso los transforma y la salida comunica resultados.
2. `print()`.
3. `input()`.
4. `str`.
5. Se produce `TypeError` por incompatibilidad entre texto y número.
6. `int()` produce enteros; `float()` produce decimales.
7. La palabra literal `nombre`.
8. Convierte un valor a texto.
9. 5 y "23", respectivamente.
10. Una cadena con `f` que inserta expresiones entre llaves.
11. Muestra dos cifras decimales.
12. `input` recibe texto; `int` intenta convertirlo; el resultado se guarda en `edad`.
13. Porque contiene un decimal y no es un entero válido.
14. Facilitan la lectura, el mantenimiento y la detección de errores.
15. Texto y número no se pueden concatenar directamente con `+`.
16. `precio = float(input("Precio: "))`.
17. Permiten interpretar correctamente el significado del valor.
18. No convertir `input()` y usar solicitudes poco claras.
19. Las variables guardan entradas y resultados que luego se muestran.
20. Solicitar primer número, convertirlo; solicitar segundo, convertirlo; sumar; mostrar.

## 16. Resumen del capítulo

Entradas y salidas son la base de la interacción entre un programa y su entorno. La entrada recibe información, el proceso trabaja con ella y la salida comunica el resultado.

| Concepto              | Para qué sirve        | Cuándo usarlo             | Error común                       |
|-----------------------|-----------------------|---------------------------|-----------------------------------|
| <code>print()</code>  | Mostrar información.  | Comunicar resultados.     | Creer que guarda datos.           |
| <code>input()</code>  | Recibir texto.        | Solicitar información.    | Olvidar <code>str</code> .        |
| <code>int()</code>    | Convertir a entero.   | Cantidades sin decimales. | Usarlo con 3.5.                   |
| <code>float()</code>  | Convertir a decimal.  | Temperaturas y costos.    | No considerar entradas inválidas. |
| <code>str()</code>    | Convertir a texto.    | Construir mensajes.       | Concatenar sin convertir.         |
| <code>f-string</code> | Formatear resultados. | Reportes claros.          | Olvidar la <code>f</code> .       |

### Ideas clave que debes recordar

- Un programa interactivo recibe, procesa y muestra.

- `print()` muestra; `input()` recibe texto.
- `input()` siempre devuelve `str`.
- Convierte antes de calcular.
- Usa `int()` para enteros y `float()` para decimales.
- Usa f-strings para mensajes claros.
- Los nombres descriptivos facilitan el mantenimiento.
- Las entradas pueden ser inválidas.
- Separar entrada, proceso y salida mejora la organización.
- Este tema prepara condicionales, ciclos y funciones.

## Mapa de aprendizaje

```

Variables y tipos de datos
    |
    v
Entradas y salidas
    |
    v
Conversiones de datos
    |
    v
Operadores y cálculos
    |
    v
Condicionales
    |
    v
Ciclos
    |
    v
Funciones
    |
    v
Proyectos completos

```

El siguiente capítulo permitirá tomar decisiones con `if`, `elif` y `else`. Gracias a las entradas y salidas, esas decisiones podrán basarse en información real.

## Verificación final

El capítulo explica qué son las entradas y salidas, por qué existen, para qué sirven, cómo funcionan, cuándo se utilizan, cómo se aplican en situaciones reales, cuáles son sus errores comunes, qué buenas prácticas seguir, cómo practicarlas y cómo integrarlas en un proyecto.